

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import MinMaxScaler,LabelEncoder
from sklearn.impute import SimpleImputer
```

```
df=pd.read_csv('/content/sample_data/TopAnimatedImDb.csv')
print('DataSet is loaded successfully')
```

DataSet is loaded successfully

```
df.shape
```

(85, 11)

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 85 entries, 0 to 84
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    Title      85 non-null     object
1    Rating     85 non-null     float64
2    Votes      85 non-null     object
3    Gross       50 non-null     object
4    Genre      85 non-null     object
5    Metascore  80 non-null     float64
6    Certificate 84 non-null     object
7    Director   85 non-null     object
8    Year       85 non-null     int64
9    Description 85 non-null     object
10   Runtime    85 non-null     object
dtypes: float64(2), int64(1), object(8)
memory usage: 7.4+ KB
```

```
df.describe()
```

	Rating	Metascore	Year
count	85.000000	80.000000	85.000000
mean	7.922353	80.850000	2003.211765
std	0.257449	8.965616	15.255830
min	7.600000	59.000000	1937.000000
25%	7.700000	74.000000	1995.000000
50%	7.900000	82.000000	2007.000000
75%	8.100000	87.000000	2014.000000
max	8.600000	96.000000	2021.000000

```
df.isnull().sum()
```

	0
Title	0
Rating	0
Votes	0
Gross	35
Genre	0
Metascore	5
Certificate	1
Director	0
Year	0
Description	0
Runtime	0

dtype: int64

```
print('No of duplicate rows:',df.duplicated().sum())
```

No of duplicate rows: 0

```
print("Rows before removing duplicates:", df.shape[0])
```

Rows before removing duplicates: 85

```
df.drop_duplicates(inplace=True)
```

```
print("Rows after removing duplicates:", df.shape[0])
```

Rows after removing duplicates: 85

```
# Check some values in Gross
print(df['Gross'].head(10))
```

```
0    $10.06M
1    $422.78M
2         NaN
3     $5.02M
4    $190.24M
5    $209.73M
6    $223.81M
7     $2.38M
8    $293.00M
9    $415.00M
```

Name: Gross, dtype: object

```
# Fill numerical columns with mean
df['Metascore'].fillna(df['Metascore'].mean(), inplace=True)
```

```
# Fill categorical column with mode
df['Certificate'].fillna(df['Certificate'].mode()[0], inplace=True)
```

/tmp/ipykernel_2349/3503650387.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chain. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values is a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df

```
df['Metascore'].fillna(df['Metascore'].mean(), inplace=True)
/tmp/ipykernel_2349/3503650387.py:5: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chain. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values is a copy.  
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df
```

```
df['Certificate'].fillna(df['Certificate'].mode()[0], inplace=True)
```

```
df.drop('Gross', axis=1, inplace=True)
print("Column 'Gross' dropped successfully.")
```

Column 'Gross' dropped successfully.

```
print(df.isnull().sum())
```

```
Title      0
Rating     0
Votes      0
Genre      0
Metascore  0
Certificate 0
Director   0
Year       0
Description 0
Runtime    0
dtype: int64
```

```
#create lablencoder
le=LabelEncoder()
#Apply labelencoding to the column sex
df['Genre']=le.fit_transform(df['Genre'])
print(df['Genre'].head())
```

```
0    5
1    4
2   17
3   16
4    0
Name: Genre, dtype: int64
```

```
#Apply one hot encoding
df=pd.get_dummies(df,columns=['Certificate'])
print(df.head())
```

```

      Title  Rating  Votes  Genre  Metascore  \
0  Sen to Chihiro no kamikakushi    8.6   747,148    5    96.0
1      The Lion King    8.5  1,041,158    4    88.0
2    Hotaru no haka    8.5   272,469   17    94.0
3    Kimi no na wa.    8.4   259,975   16    79.0
4  Spider-Man: Into the Spider-Verse    8.4   510,227    0    87.0

      Director  Year  Description  \
0  Hayao Miyazaki  2001  ["\nDuring her family's move to the suburbs, a...
1  Roger Allers  1994  ["\nLion prince Simba and his father are targe...
2  Isao Takahata  1988  ["\nA young boy and his little sister struggle...
3  Makoto Shinkai  2016  ["\nTwo strangers find themselves linked in a ...
4  Bob Persichetti  2018  ["\nTeen Miles Morales becomes the Spider-Man ...

      Runtime  Certificate_13  Certificate_16  Certificate_7  Certificate_A  \
0   125 min         False         False         False         False
1    88 min         False         False         False         False
2    89 min         False         False         False         False
3   106 min         False         False         False         False
4   117 min         False         False         False         False

      Certificate_Not  Rated  Certificate_PG  Certificate_PG-13  Certificate_R  \
0                False    False         False         False    False
1                False    False         False         False    False
2                False    False         False         False    False
3                False    False         False         False    False
4                False    False         False         False    False

      Certificate_U  Certificate_UA
0                True         False
1                True         False
2                True         False
3                True         False
4                True         False
```

```
#create MinMaxScaler
scaler=MinMaxScaler()
#Normalize Age and Fare columns
df[['Rating','Metascore']]=scaler.fit_transform(df[['Rating','Metascore']])
print(df[['Rating','Metascore']].head())
```

```
Rating  Metascore
0      1.0    1.000000
1      0.9    0.783784
```

```
2    0.9    0.945946
3    0.8    0.540541
4    0.8    0.756757
```

```
# Create a new feature 'MovieAge'
df['MovieAge'] = 2026 - df['Year']
print("MovieAge feature Created")
display(df[['Year', 'MovieAge']].head())
```

MovieAge feature Created

	Year	MovieAge
0	2001	25
1	1994	32
2	1988	38
3	2016	10
4	2018	8

```
# Create a new feature 'HighRated' where 1 indicates Rating >= 8.0
df['HighRated'] = df['Rating'].apply(lambda x: 1 if x >= 8.0 else 0)
print("HighRated feature Created")
display(df[['Rating', 'HighRated']].head())
```

HighRated feature Created

	Rating	HighRated
0	1.0	0
1	0.9	0
2	0.9	0
3	0.8	0
4	0.8	0

```
# Outlier detection
Q1 = df['Rating'].quantile(0.25)
Q3 = df['Rating'].quantile(0.75)
IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

df = df[(df['Rating'] >= lower_limit) & (df['Rating'] <= upper_limit)]

print("Outliers removed")
```

Outliers removed

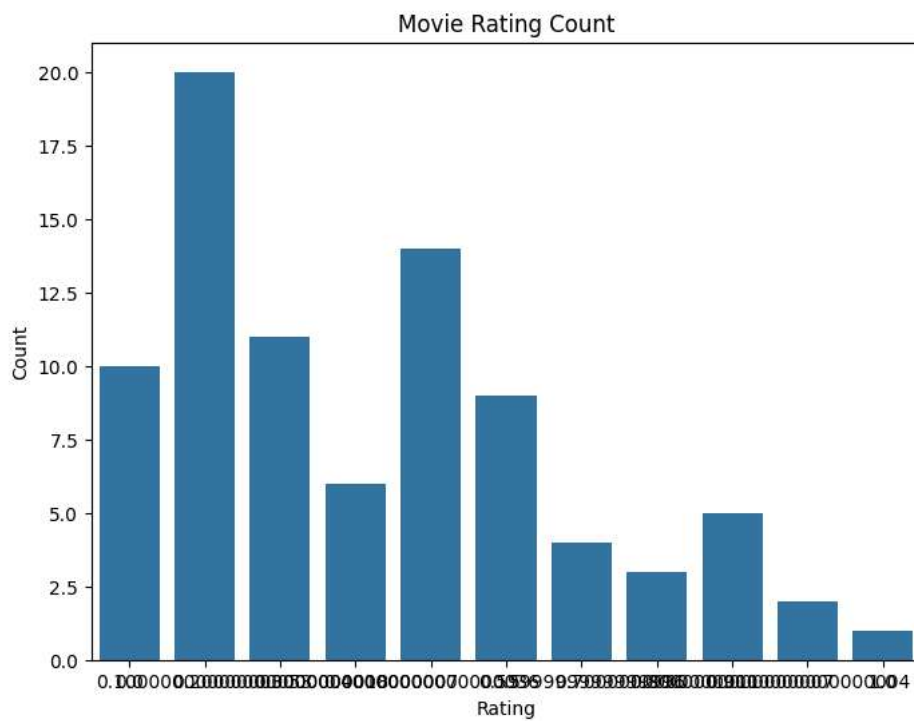
```
print("Dataset shape After removing outliers:", df.shape)
```

Dataset shape After removing outliers: (85, 21)

```
# Univariate Visualization
plt.figure(figsize=(8,6))
sns.countplot(x='Rating', data=df)

plt.title('Movie Rating Count')
plt.xlabel('Rating')
plt.ylabel('Count')

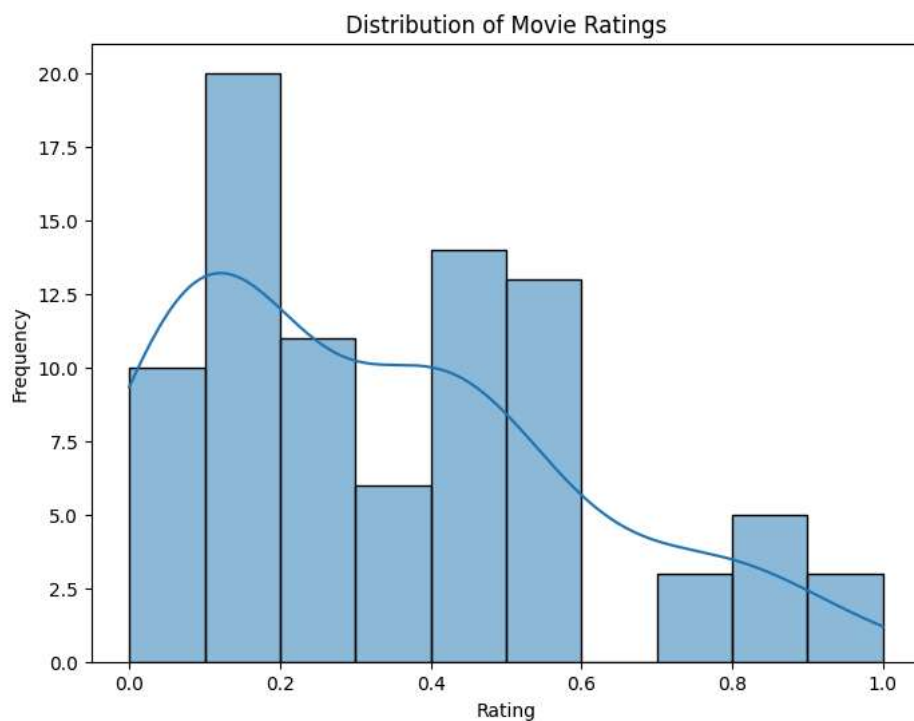
plt.show()
```



```
# Plot the distribution of movie ratings
plt.figure(figsize=(8,6))
sns.histplot(df['Rating'], bins=10, kde=True)

plt.title('Distribution of Movie Ratings')
plt.xlabel('Rating')
plt.ylabel('Frequency')

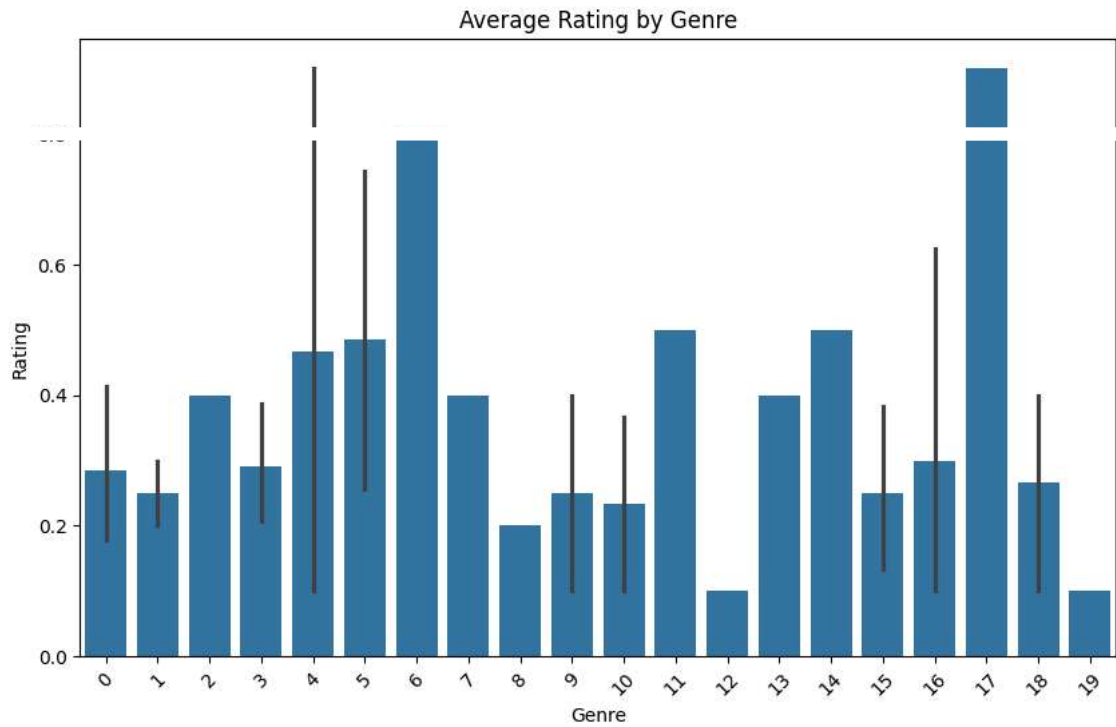
plt.show()
```



```
plt.figure(figsize=(10,6))
sns.barplot(x='Genre', y='Rating', data=df)

plt.title('Average Rating by Genre')
plt.xlabel('Genre')
plt.ylabel('Rating')
```

```
plt.xticks(rotation=45)
plt.show()
```



```
# Drop columns 'Description' and 'Title'
drop_columns = ['Description', 'Title']

df.drop(drop_columns, axis=1, inplace=True)

print("Columns dropped successfully")
```

Columns dropped successfully

```
print("After preprocessing")
display(df.head(10))
```

After preprocessing

	Rating	Votes	Genre	Metascore	Director	Year	Runtime	Certificate_13	Certificate_16	Certificate_7	Certificate_A	Cer
0	1.0	747,148	5	1.000000	Hayao Miyazaki	2001	125 min	False	False	False	False	
1	0.9	1,041,158	4	0.783784	Roger Allers	1994	88 min	False	False	False	False	
2	0.9	272,469	17	0.945946	Isao Takahata	1988	89 min	False	False	False	False	
3	0.8	259,975	16	0.540541	Makoto Shinkai	2016	106 min	False	False	False	False	
4	0.8	510,227	0	0.756757	Bob Persichetti	2018	117 min	False	False	False	False	
5	0.8	492,081	3	0.594595	Lee Unkrich	2017	105 min	False	False	False	False	
6	0.8	1,100,175	5	0.972973	Andrew Stanton	2008	98 min	False	False	False	False	
7	0.8	388,925	6	0.459459	Hayao Miyazaki	1997	134 min	False	False	False	False	